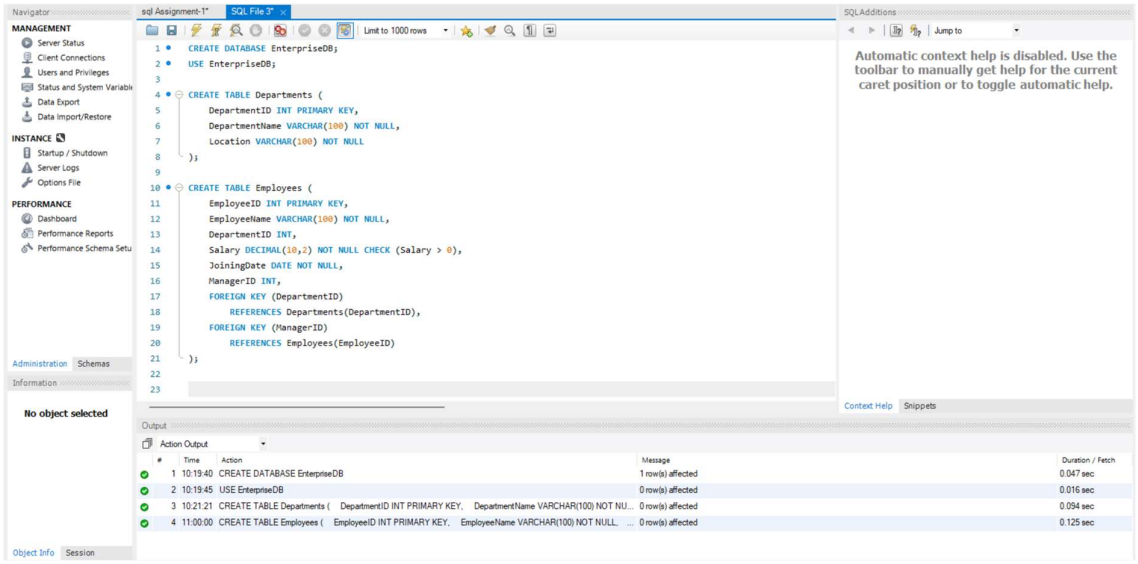
Name : M.Lokeshwar Reddy
USN : 23BTRCL217
ID : HCLTFP2283087
Collage Name : Jain University

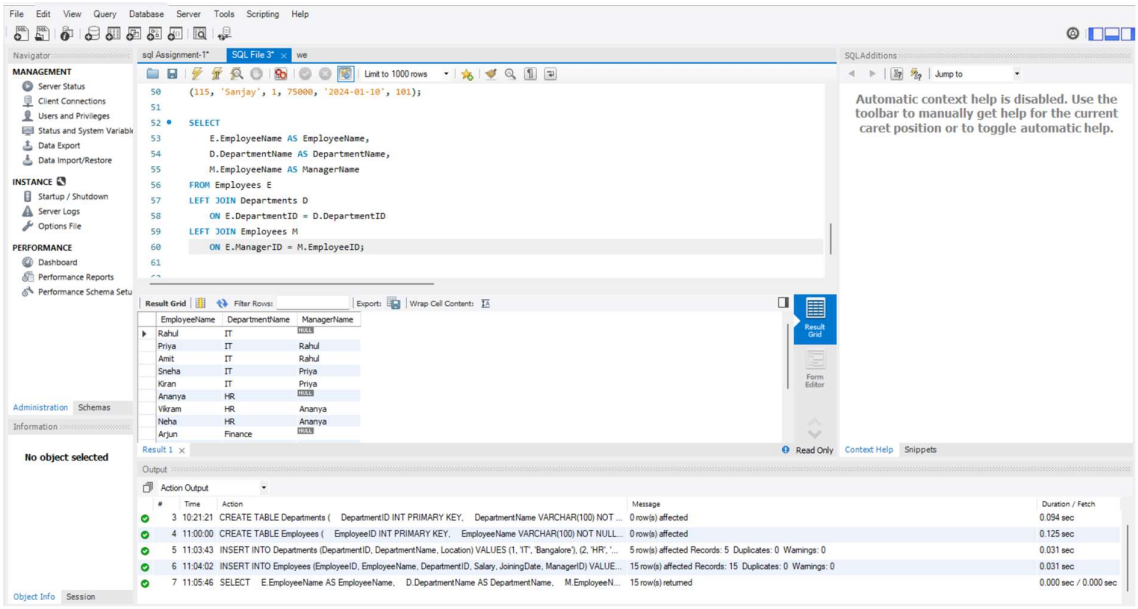
Question 1: Database Design and Advanced Queries

Create a database named EnterpriseDB.

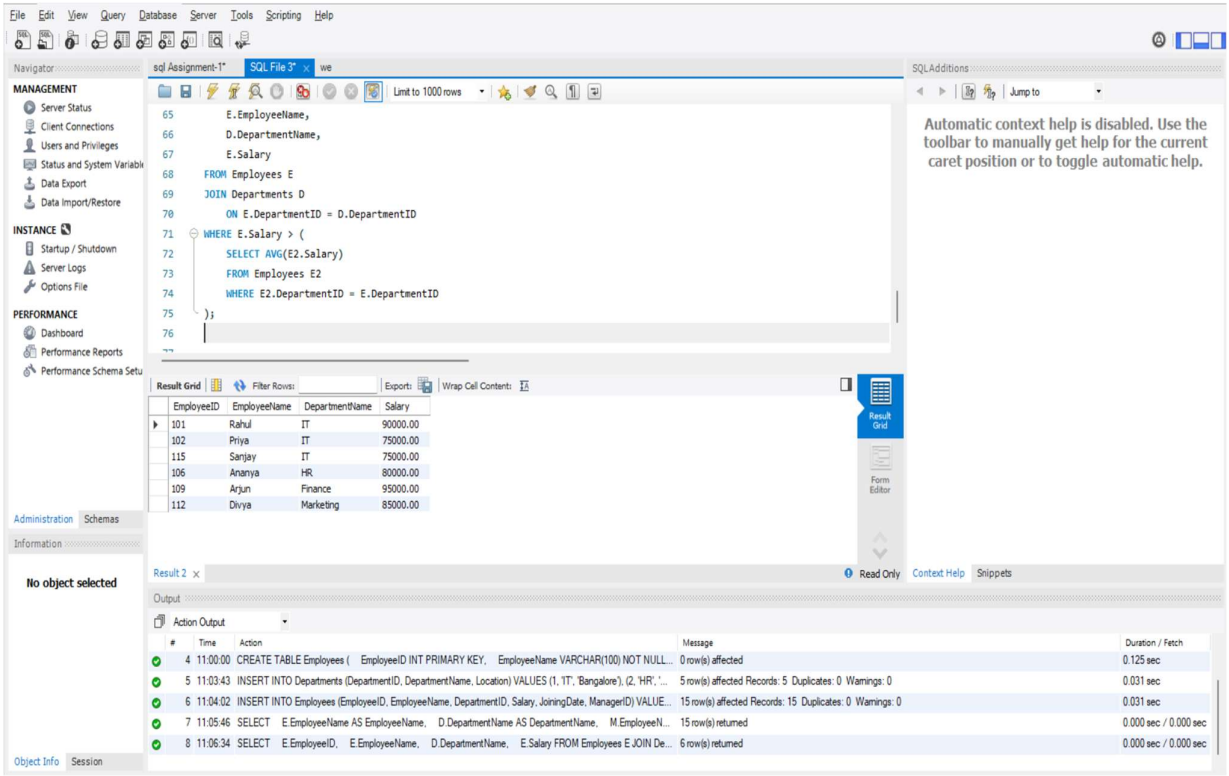
Create the following tables.



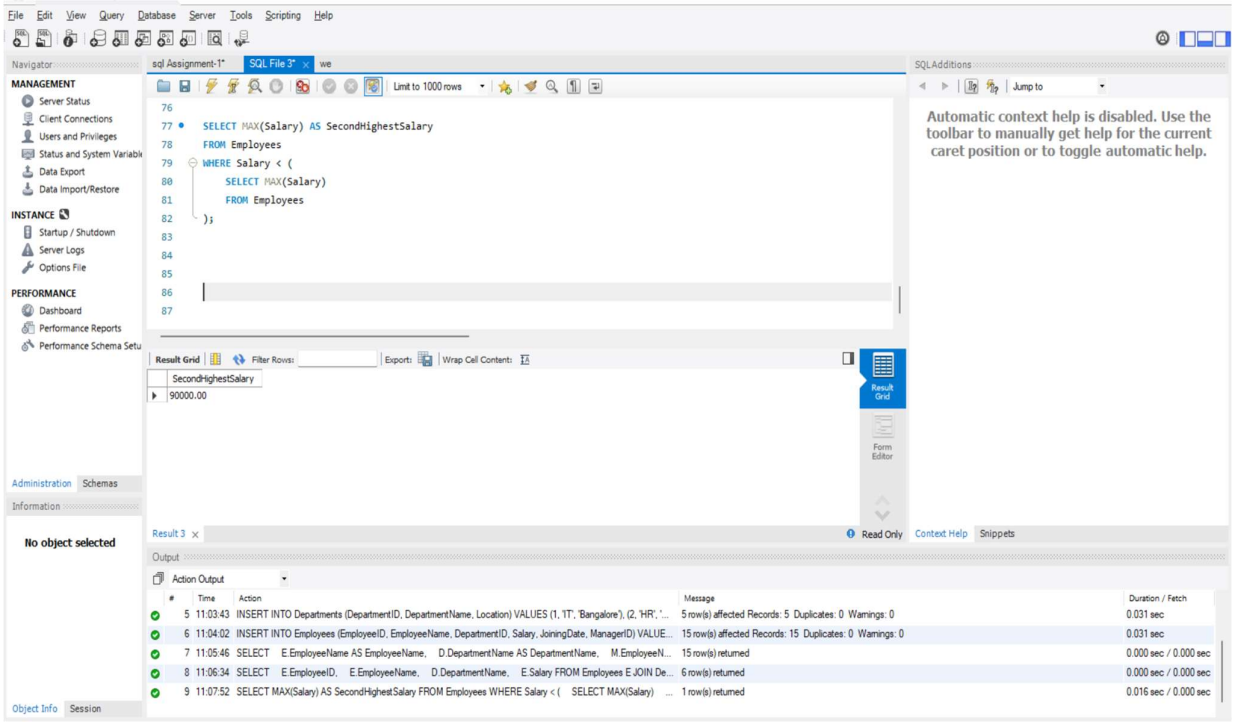
a. Display employee names, department names, and manager names using joins.



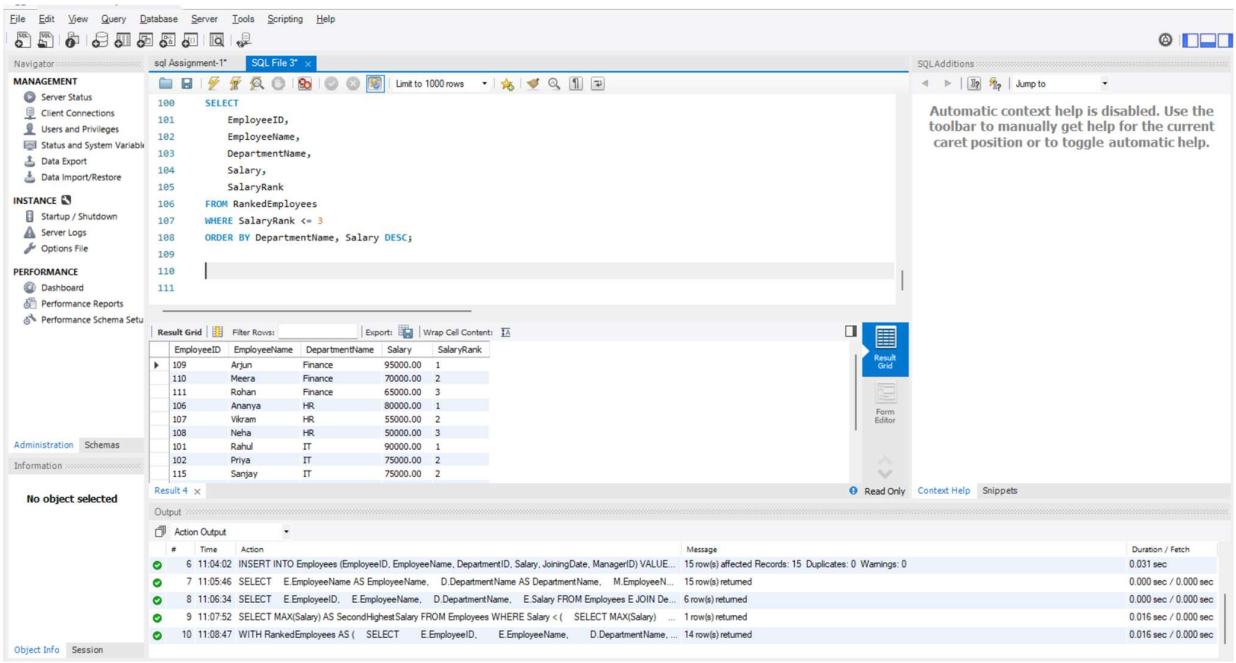
b. Find employees earning more than their department's average salary.



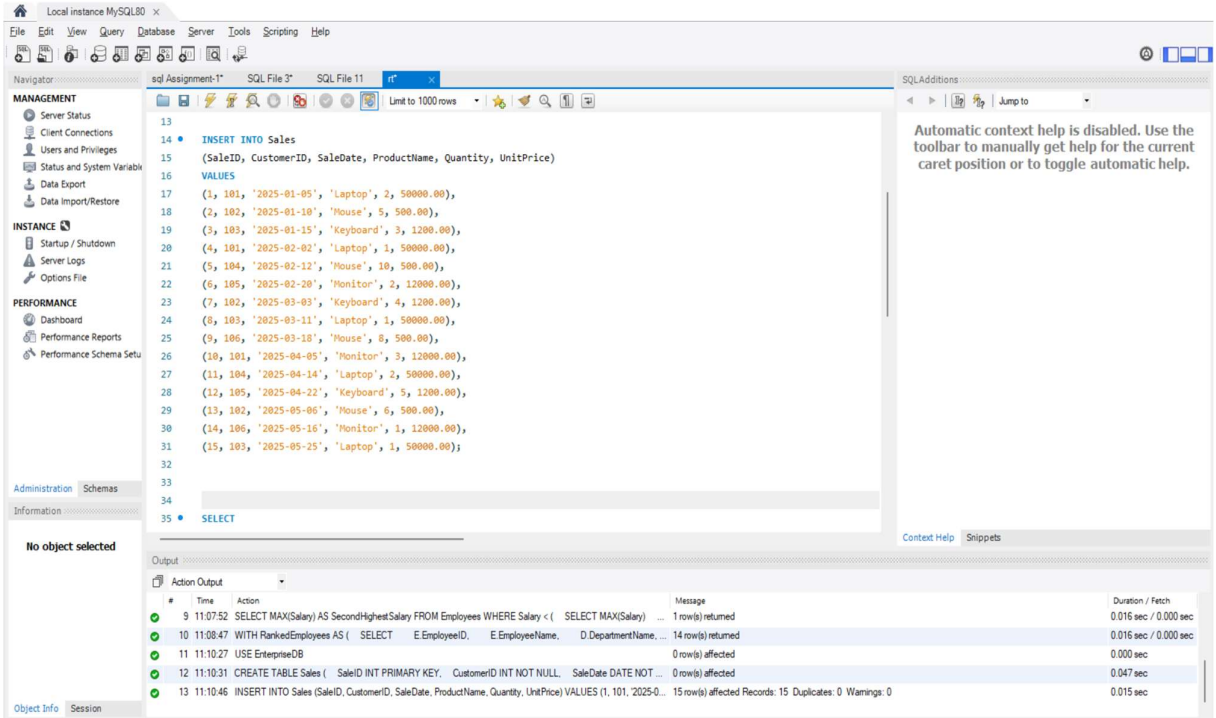
c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST.



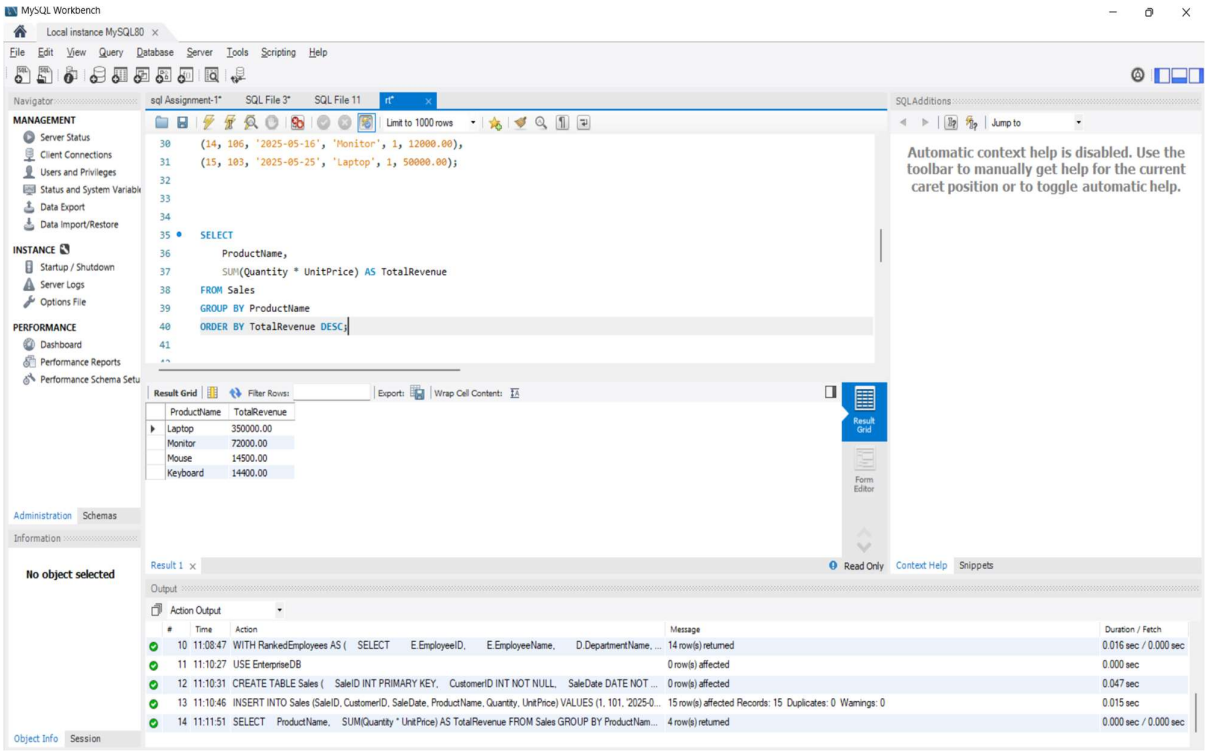
d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff.



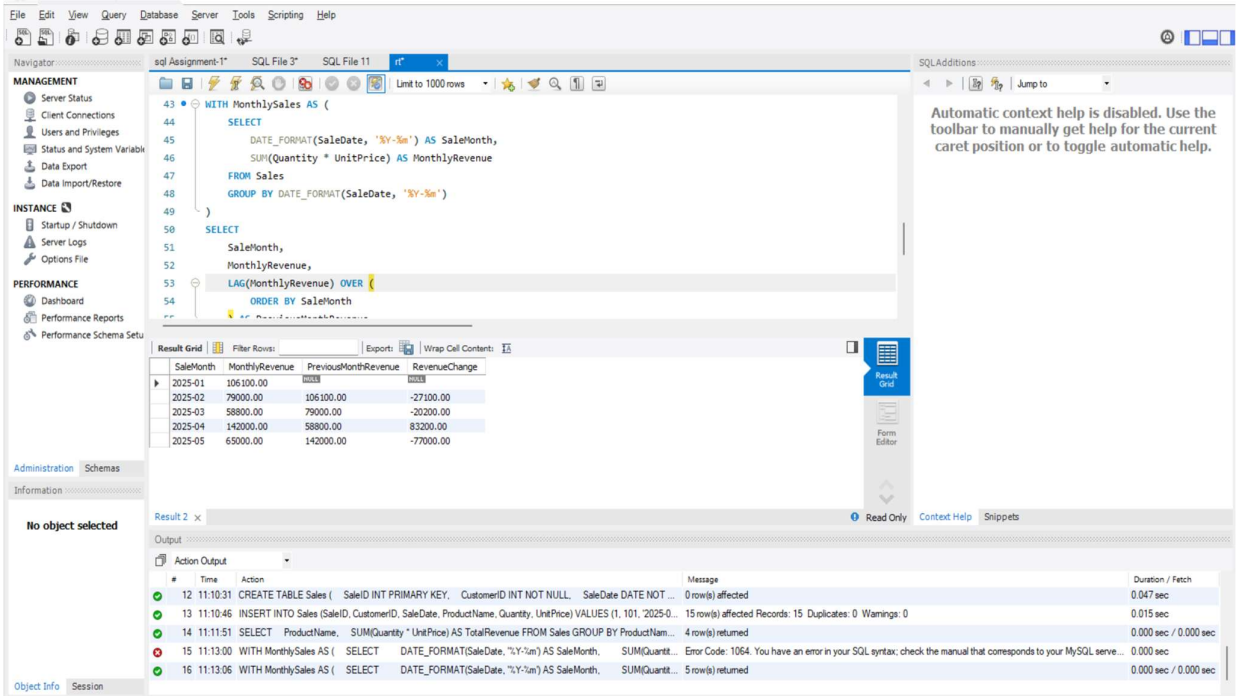
Question 2: Sales Analytics and Query Optimization



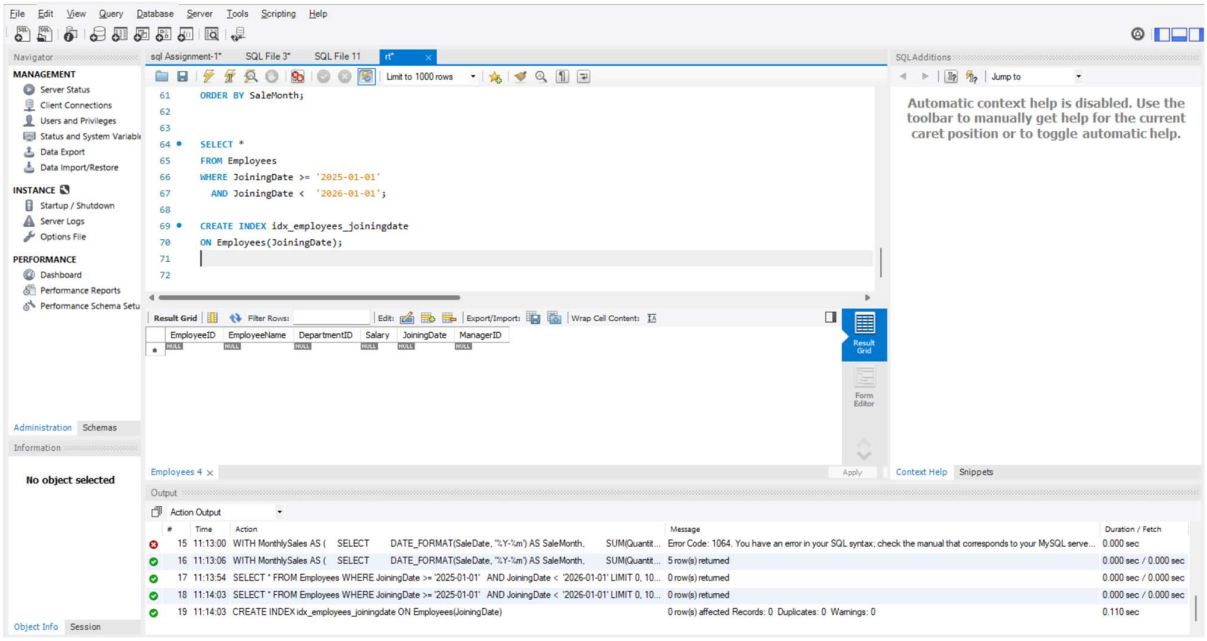
a. Calculate total revenue for each product.



b. Calculate monthly revenue and month-over-month revenue change using a window function.



c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better.



Question 3: SQL Troubleshooting

a. Explain how an execution plan helps identify expensive operations in a query.

A. An execution plan shows how SQL runs a query and helps find slow or costly operations like full table scans.

b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning.

A. A composite index on (DepartmentID, Salary) speeds up queries filtering by both columns by reducing the number of rows scanned.

c. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once.

A. GROUP BY groups identical email addresses, and HAVING COUNT(*) > 1 displays only addresses that occur more than once.

Question 4: Log Analysis

a. grep "ERROR" /var/log/application.log

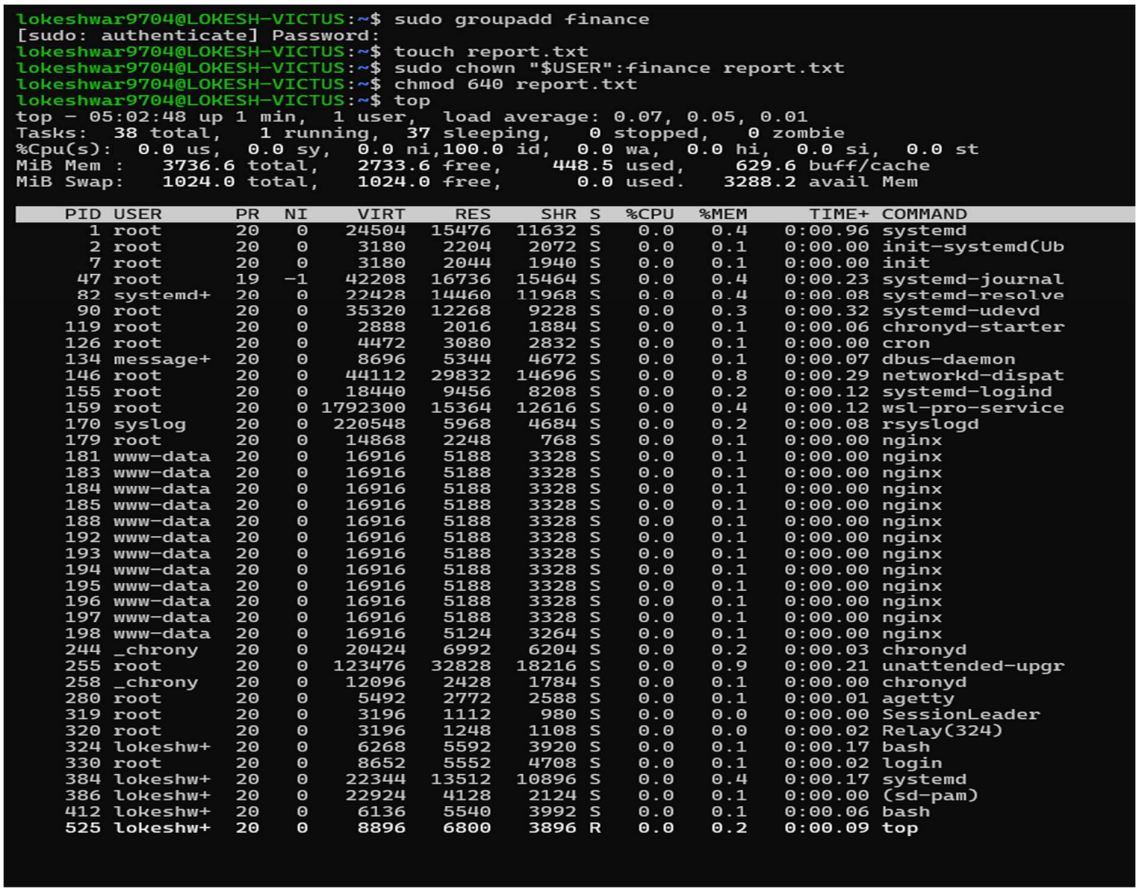
b. grep "ERROR" /var/log/application.log | tail -n 20

c. grep -c "ERROR" /var/log/application.log

d. grep "^2026-10-28.*ERROR" /var/log/application.log > /var/log/errors-2026-10-28.log

e. grep "ERROR" /var/log/application.log | awk '{print substr($2,1,2)}' | sort | uniq -c

Question 5: Permissions and System Troubleshooting



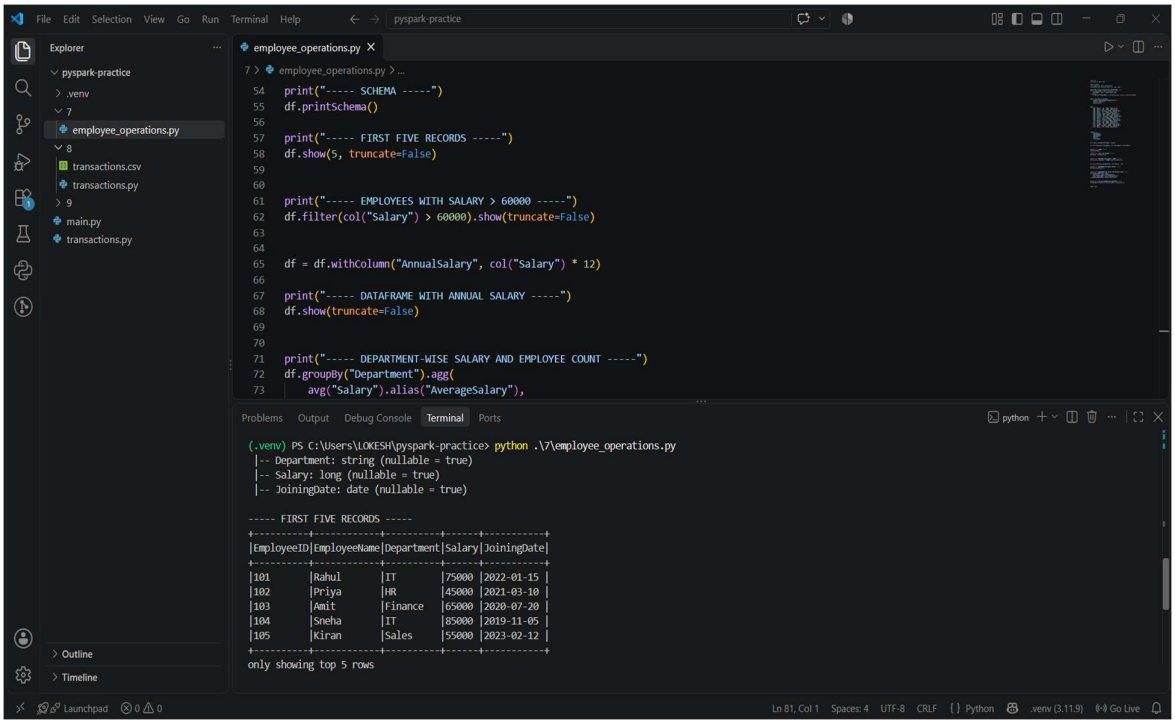
```
lokeshwar9704@LOKESH-VICTUS:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.6Gi       444Mi       2.7Gi       3.5Mi       629Mi       3.2Gi
Swap:          1.0Gi          0B       1.0Gi
lokeshwar9704@LOKESH-VICTUS:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            1.9G     0  1.9G   0% /usr/lib/modules/6.18.33.2-microsoft-standard-WSL2
none            1.9G  4.0K  1.9G   1% /mnt/wsl
drivers         122G  110G   12G  91% /usr/lib/wsl/drivers
/dev/sdd       1007G  1.7G  955G   1% /
none            1.9G   32K  1.9G   1% /mnt/wslg
none            1.9G     0  1.9G   0% /usr/lib/wsl/lib
rootfs          1.9G  2.8M  1.9G   1% /init
none            1.9G  536K  1.9G   1% /run
none            1.9G     0  1.9G   0% /run/lock
none            1.9G     0  1.9G   0% /run/shm
none            1.9G   80K  1.9G   1% /mnt/wslg/versions.txt
none            1.9G   80K  1.9G   1% /mnt/wslg/doc
C:\             122G  110G   12G  91% /mnt/c
D:\             355G   41G  315G  12% /mnt/d
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           1.9G     0  1.9G   0% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
none            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
tmpfs           374M   12K  374M   1% /run/user/1000
lokeshwar9704@LOKESH-VICTUS:~$ find /opt/app -size +100M
find: ‘/opt/app’: No such file or directory
lokeshwar9704@LOKESH-VICTUS:~$
```

Question 6: Bash Scripting

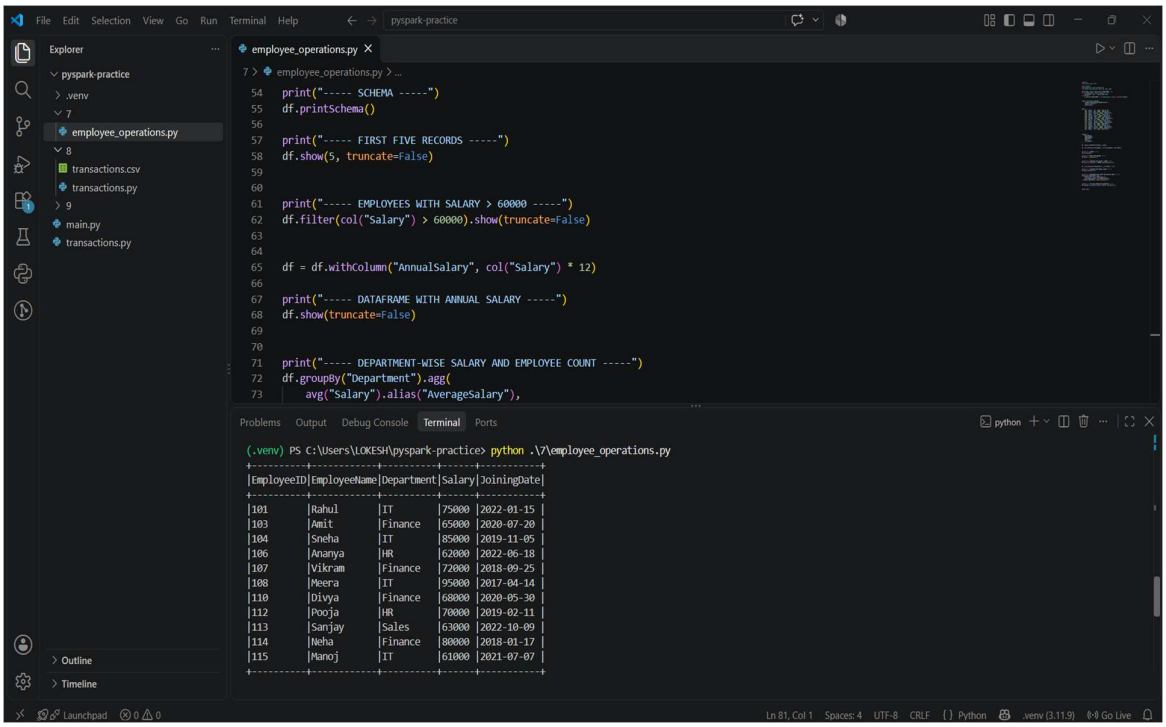
```
lokeshwar9704@LOKESH-VIC
lokeshwar9704@LOKESH-VICTUS:~$ nano backup_check.sh
lokeshwar9704@LOKESH-VICTUS:~$ chmod +x backup_check.sh
lokeshwar9704@LOKESH-VICTUS:~$ mkdir source
lokeshwar9704@LOKESH-VICTUS:~$ ./backup_check.sh source backup
Backup successful
lokeshwar9704@LOKESH-VICTUS:~$ ./backup_check.sh source
Error: Give two arguments
lokeshwar9704@LOKESH-VICTUS:~$ ./backup_check.sh xyz backup
Source directory does not exist
lokeshwar9704@LOKESH-VICTUS:~$
```

Question 7: PySpark DataFrame Operations

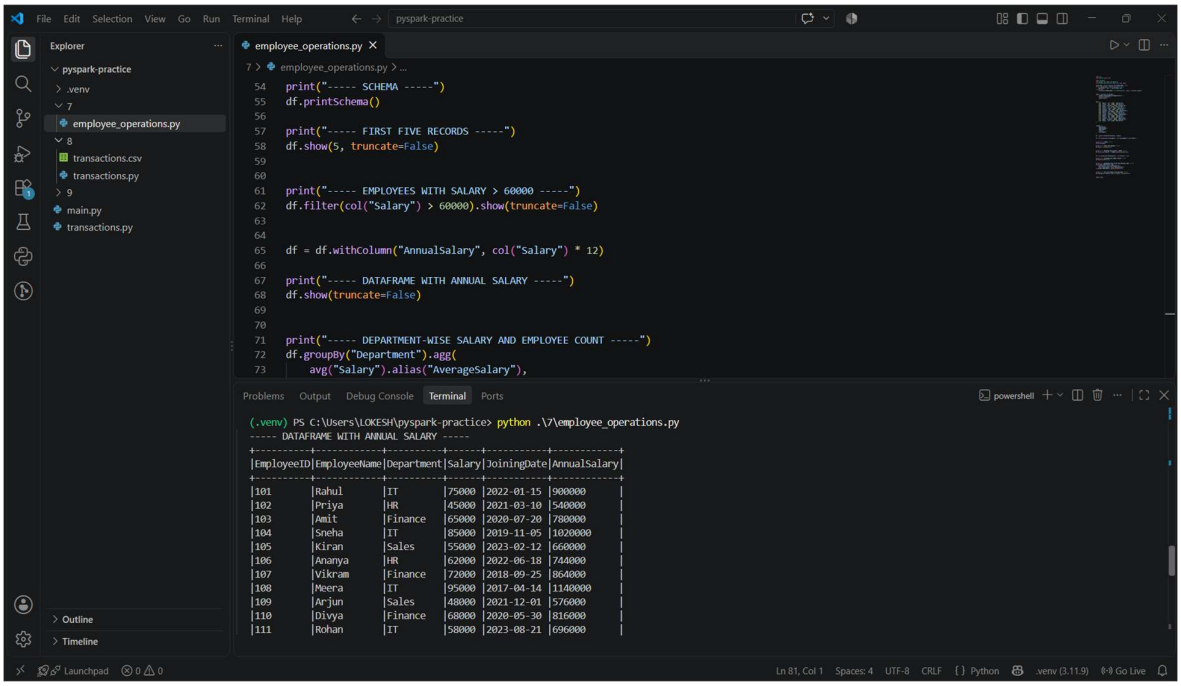
a. Display the schema and first five records.



b. Filter employees whose monthly salary is greater than ₹60,000.



c. Add an AnnualSalary column calculated from the monthly salary.



```
print("----- SCHEMA -----")
df.printSchema()

print("----- FIRST FIVE RECORDS -----")
df.show(5, truncate=False)


print("----- EMPLOYEES WITH SALARY > 60000 -----")
df.filter(col("Salary") > 60000).show(truncate=False)

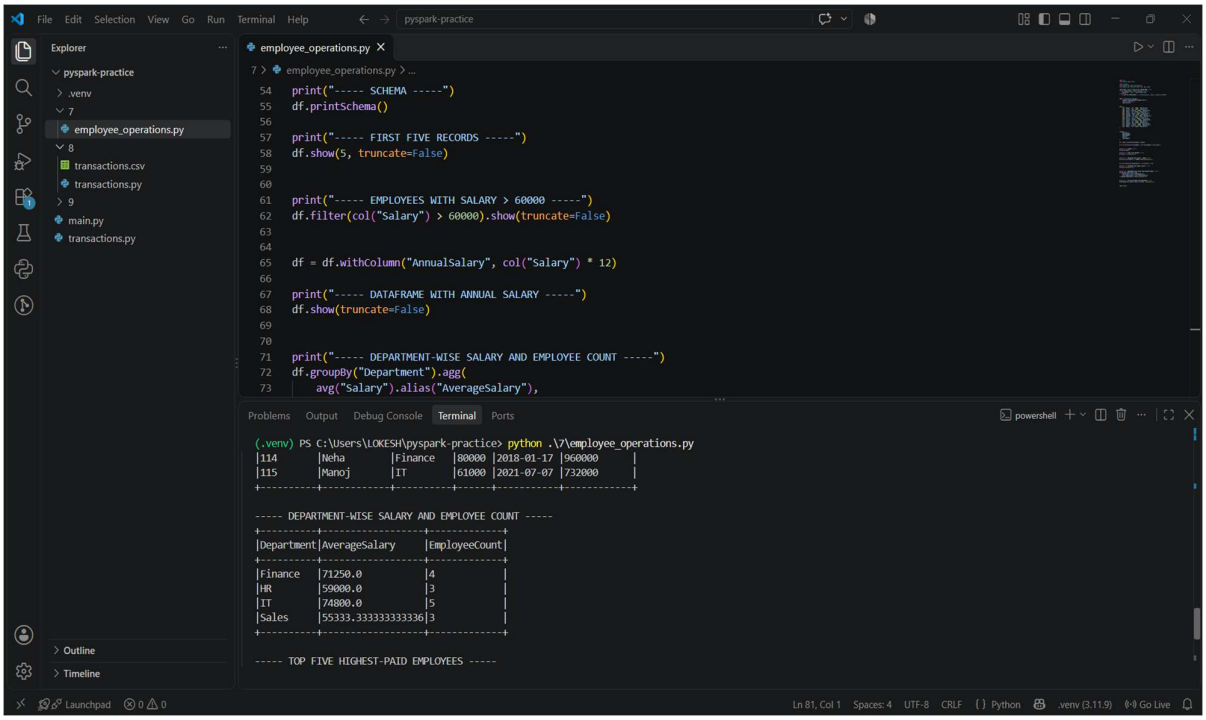

df = df.withColumn("AnnualSalary", col("Salary") * 12)

print("----- DATAFRAME WITH ANNUAL SALARY -----")
df.show(truncate=False)


print("----- DEPARTMENT-WISE SALARY AND EMPLOYEE COUNT -----")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
```

```
(.venv) PS C:\Users\LOKESH\pyspark-practice> python .\7\employee_operations.py
----- DATAFRAME WITH ANNUAL SALARY -----
+----------+------------+----------+------+----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+----------+------------+
|101       |Rahul       |IT        |75000 |2022-01-15 |900000      |
|102       |Priya       |HR        |45000 |2021-03-10 |540000      |
|103       |Amit        |Finance   |65000 |2020-07-20 |780000      |
|104       |Sneha       |IT        |85000 |2019-11-05 |1020000     |
|105       |Kiran       |Sales     |55000 |2023-02-12 |660000      |
|106       |Ananya      |HR        |62000 |2022-06-18 |744000      |
|107       |Vikram      |Finance   |72000 |2018-09-25 |864000      |
|108       |Meera       |IT        |95000 |2017-04-14 |1140000     |
|109       |Arjun       |Sales     |48000 |2021-12-01 |576000      |
|110       |Divya       |Finance   |68000 |2020-05-30 |816000      |
|111       |Rohan       |IT        |58000 |2023-08-21 |696000      |
```

d. Group employees by department and calculate the average salary and employee count.



```
print("----- SCHEMA -----")
df.printSchema()

print("----- FIRST FIVE RECORDS -----")
df.show(5, truncate=False)


print("----- EMPLOYEES WITH SALARY > 60000 -----")
df.filter(col("Salary") > 60000).show(truncate=False)


df = df.withColumn("AnnualSalary", col("Salary") * 12)

print("----- DATAFRAME WITH ANNUAL SALARY -----")
df.show(truncate=False)

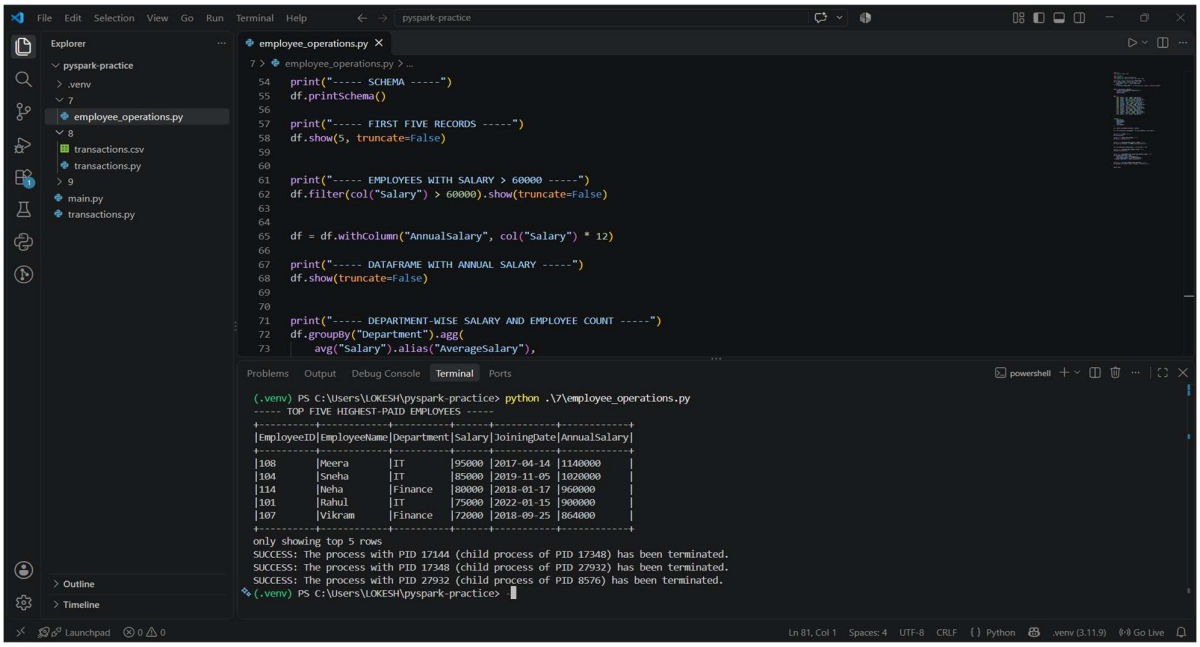

print("----- DEPARTMENT-WISE SALARY AND EMPLOYEE COUNT -----")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
```

```
(.venv) PS C:\Users\LOKESH\pyspark-practice> python .\7\employee_operations.py
|114       |Neha        |Finance   |80000 |2018-01-17 |960000      |
|115       |Manoj       |IT        |61000 |2021-07-07 |732000      |
+----------+------------+----------+------+----------+------------+

----- DEPARTMENT-WISE SALARY AND EMPLOYEE COUNT -----
+----------+------------------+-------------+
|Department|AverageSalary     |EmployeeCount|
+----------+------------------+-------------+
|Finance   |71250.0           |4            |
|HR        |59000.0           |3            |
|IT        |74800.0           |5            |
|Sales     |55333.333333333336|3            |
+----------+------------------+-------------+

----- TOP FIVE HIGHEST-PAID EMPLOYEES -----
```

e. Display the five highest-paid employees.



```
print("----- SCHEMA -----")
df.printSchema()

print("----- FIRST FIVE RECORDS -----")
df.show(5, truncate=False)


print("----- EMPLOYEES WITH SALARY > 60000 -----")
df.filter(col("Salary") > 60000).show(truncate=False)


df = df.withColumn("AnnualSalary", col("Salary") * 12)

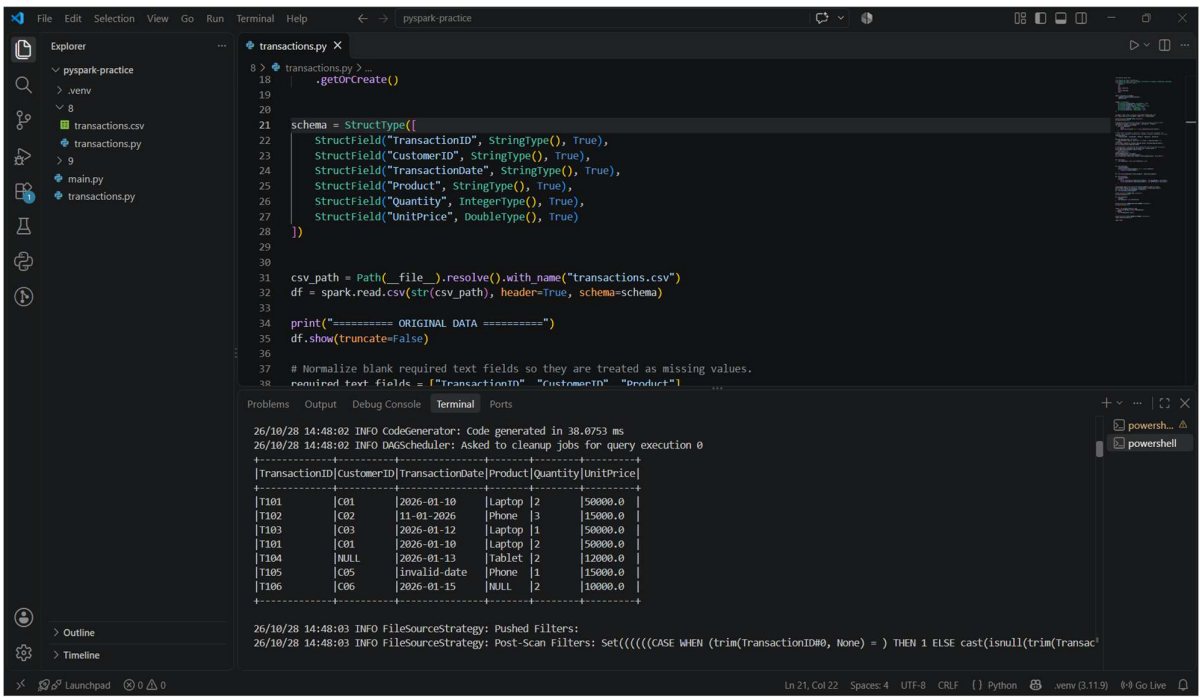
print("----- DATAFRAME WITH ANNUAL SALARY -----")
df.show(truncate=False)


print("----- DEPARTMENT-WISE SALARY AND EMPLOYEE COUNT -----")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
```

```
(.venv) PS C:\Users\LOKESH\pyspark-practice> python .\7\employee_operations.py
----- TOP FIVE HIGHEST-PAID EMPLOYEES -----
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|108       |Meera       |IT        |95000 |2017-04-14 |1140000     |
|104       |Sneha       |IT        |85000 |2019-11-05 |1020000     |
|114       |Neha        |Finance   |80000 |2018-01-17 |960000      |
|101       |Rahul       |IT        |75000 |2022-01-15 |900000      |
|107       |Vikram      |Finance   |72000 |2018-09-25 |864000      |
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
SUCCESS: The process with PID 17144 (child process of PID 17348) has been terminated.
SUCCESS: The process with PID 17348 (child process of PID 27932) has been terminated.
SUCCESS: The process with PID 27932 (child process of PID 8576) has been terminated.
(.venv) PS C:\Users\LOKESH\pyspark-practice>
```

Question 8: Data Cleaning and Aggregation

a. Read the CSV file using an appropriate schema and header configuration.



```
    .getOrCreate()


schema = StructType([
    StructField("TransactionID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("TransactionDate", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True)
])


csv_path = Path(__file__).resolve().with_name("transactions.csv")
df = spark.read.csv(str(csv_path), header=True, schema=schema)

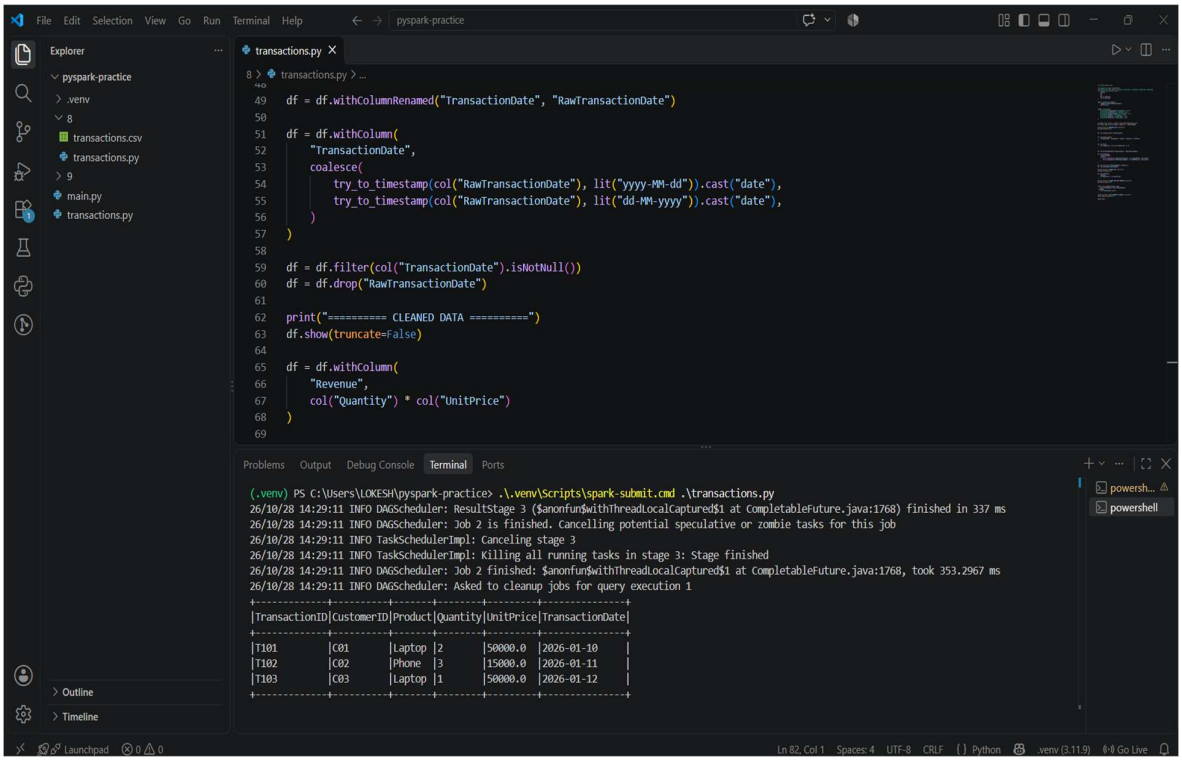
print("========== ORIGINAL DATA ==========")
df.show(truncate=False)

# Normalize blank required text fields so they are treated as missing values.
required_text_fields = ["TransactionID", "CustomerID", "Product"]
```

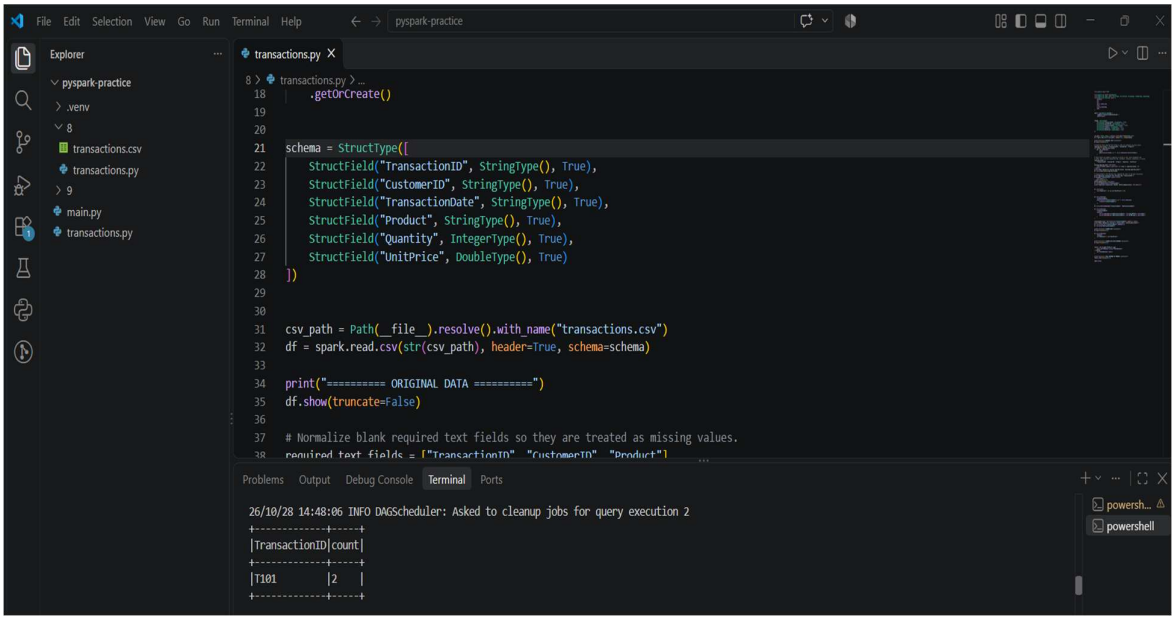
```
26/10/28 14:48:02 INFO CodeGenerator: Code generated in 38.0753 ms
26/10/28 14:48:02 INFO DAGScheduler: Asked to cleanup jobs for query execution 0
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|T101         |C01       |2026-01-10     |Laptop |2       |50000.0  |
|T102         |C02       |11-01-2026     |Phone  |3       |15000.0  |
|T103         |C03       |2026-01-12     |Laptop |1       |50000.0  |
|T101         |C01       |2026-01-10     |Laptop |2       |50000.0  |
|T104         |NULL      |2026-01-13     |Tablet |2       |12000.0  |
|T105         |C05       |invalid-date   |Phone  |1       |15000.0  |
|T106         |C06       |2026-01-15     |NULL   |2       |10000.0  |
+-------------+----------+---------------+-------+--------+---------+

26/10/28 14:48:03 INFO FileSourceStrategy: Pushed Filters:
26/10/28 14:48:03 INFO FileSourceStrategy: Post-Scan Filters: Set((((((CASE WHEN (trim(TransactionID#0, None) = ) THEN 1 ELSE cast(isnull(trim(Transac
```

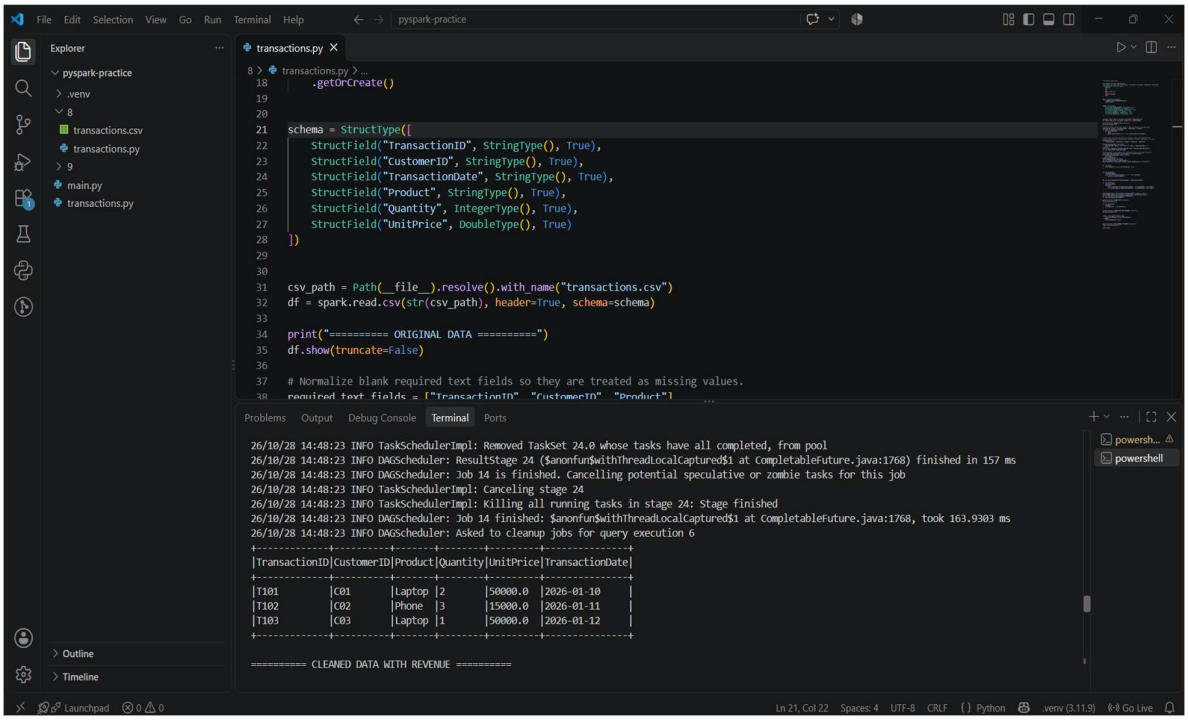
b. Identify and remove duplicate transactions using a suitable business key.



c. Handle missing values in required fields and explain your chosen approach.



d. Convert TransactionDate into a date type and handle invalid dates.



```
    .getOrCreate()


schema = StructType([
    StructField("TransactionID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("TransactionDate", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True)
])


csv_path = Path(__file__).resolve().with_name("transactions.csv")
df = spark.read.csv(str(csv_path), header=True, schema=schema)

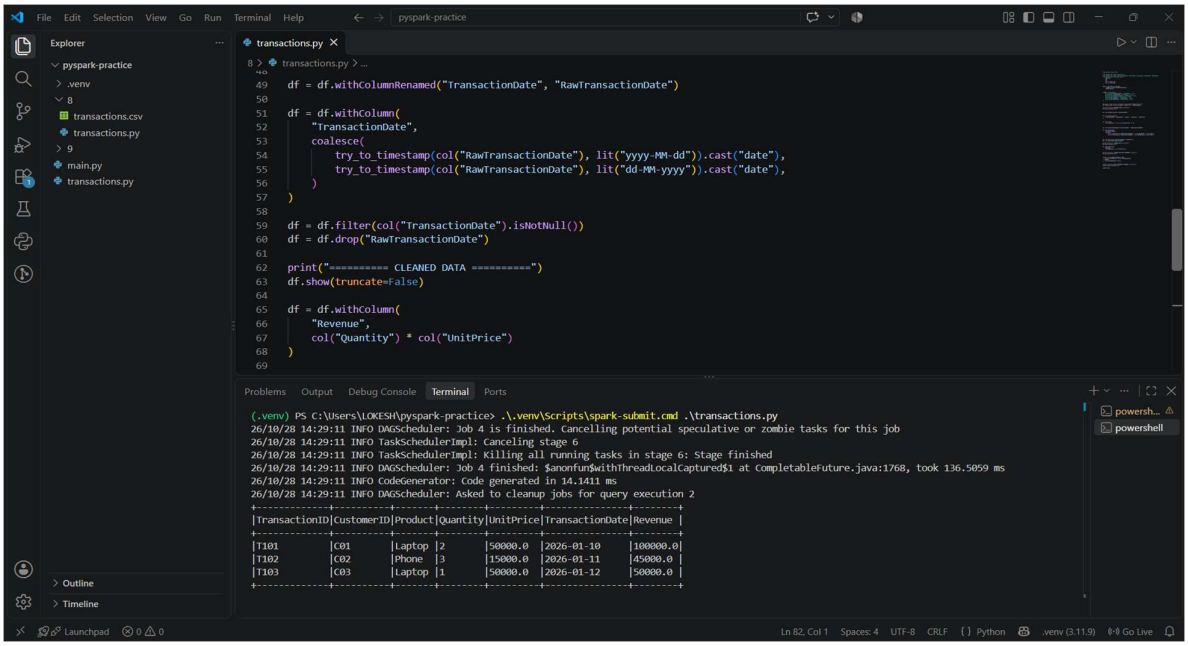
print("========== ORIGINAL DATA ==========")
df.show(truncate=False)

# Normalize blank required text fields so they are treated as missing values.
```

```
26/10/28 14:48:23 INFO TaskSchedulerImpl: Removed TaskSet 24.0 whose tasks have all completed, from pool
26/10/28 14:48:23 INFO DAGScheduler: ResultStage 24 ($anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768) finished in 157 ms
26/10/28 14:48:23 INFO DAGScheduler: Job 14 is finished. Cancelling potential speculative or zombie tasks for this job
26/10/28 14:48:23 INFO TaskSchedulerImpl: Canceling stage 24
26/10/28 14:48:23 INFO TaskSchedulerImpl: Killing all running tasks in stage 24: Stage finished
26/10/28 14:48:23 INFO DAGScheduler: Job 14 finished: $anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768, took 163.9303 ms
26/10/28 14:48:23 INFO DAGScheduler: Asked to cleanup jobs for query execution 6
+-------------+----------+-------+--------+---------+---------------+
|TransactionID|CustomerID|Product|Quantity|UnitPrice|TransactionDate|
+-------------+----------+-------+--------+---------+---------------+
|T101         |C01       |Laptop |2       |50000.0  |2026-01-10     |
|T102         |C02       |Phone  |3       |15000.0  |2026-01-11     |
|T103         |C03       |Laptop |1       |50000.0  |2026-01-12     |
+-------------+----------+-------+--------+---------+---------------+

========== CLEANED DATA WITH REVENUE ==========
```

e. Calculate revenue using Quantity × UnitPrice.



```
df = df.withColumnRenamed("TransactionDate", "RawTransactionDate")

df = df.withColumn(
    "TransactionDate",
    coalesce(
        try_to_timestamp(col("RawTransactionDate"), lit("yyyy-MM-dd")).cast("date"),
        try_to_timestamp(col("RawTransactionDate"), lit("dd-MM-yyyy")).cast("date"),
    )
)

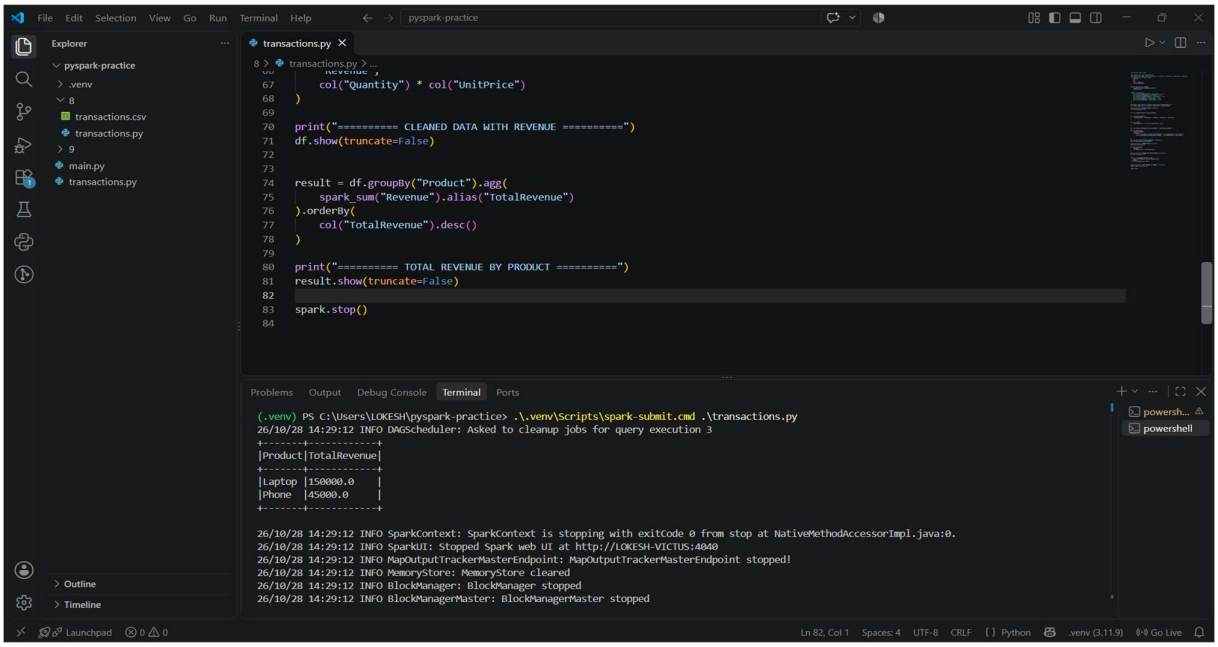
df = df.filter(col("TransactionDate").isNotNull())
df = df.drop("RawTransactionDate")

print("========== CLEANED DATA ==========")
df.show(truncate=False)

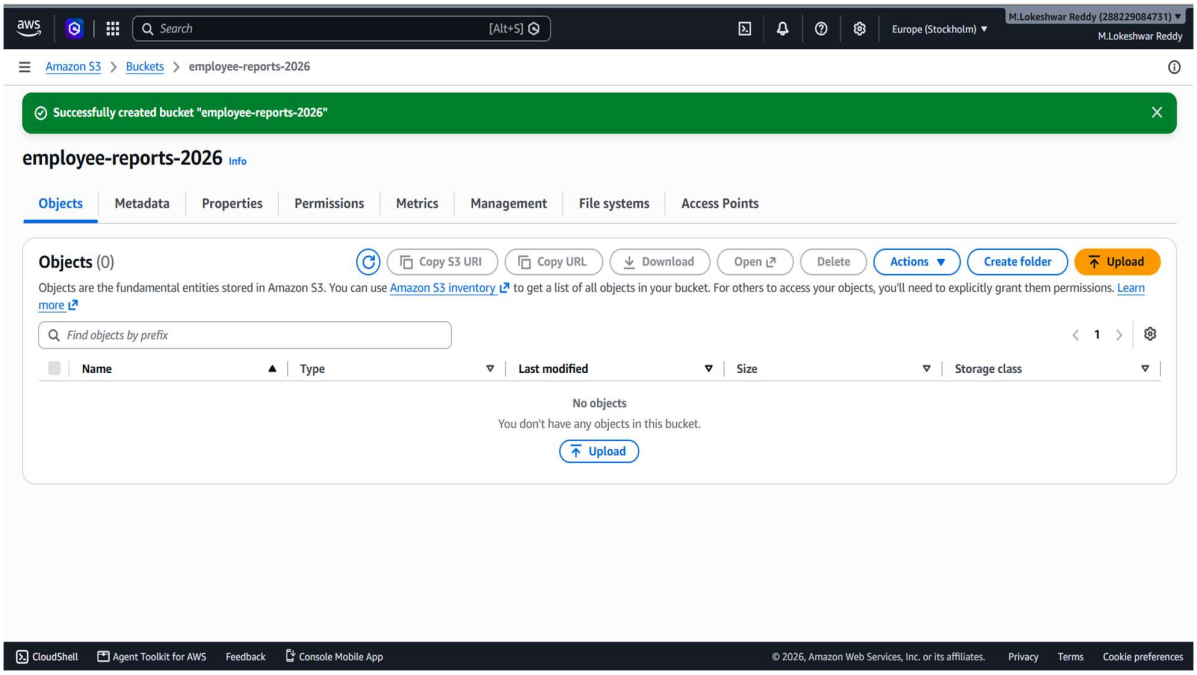
df = df.withColumn(
    "Revenue",
    col("Quantity") * col("UnitPrice")
)
```

```
(.venv) PS C:\Users\LOKESH\pyspark-practice> .\.venv\Scripts\spark-submit.cmd .\transactions.py
26/10/28 14:29:11 INFO DAGScheduler: Job 4 is finished. Cancelling potential speculative or zombie tasks for this job
26/10/28 14:29:11 INFO TaskSchedulerImpl: Canceling stage 6
26/10/28 14:29:11 INFO TaskSchedulerImpl: Killing all running tasks in stage 6: Stage finished
26/10/28 14:29:11 INFO DAGScheduler: Job 4 finished: $anonfun$withThreadLocalCaptured$1 at CompletableFuture.java:1768, took 136.5059 ms
26/10/28 14:29:11 INFO CodeGenerator: Code generated in 14.1411 ms
26/10/28 14:29:11 INFO DAGScheduler: Asked to cleanup jobs for query execution 2
+-------------+----------+-------+--------+---------+---------------+-------+
|TransactionID|CustomerID|Product|Quantity|UnitPrice|TransactionDate|Revenue |
+-------------+----------+-------+--------+---------+---------------+-------+
|T101         |C01       |Laptop |2       |50000.0  |2026-01-10     |100000.0|
|T102         |C02       |Phone  |3       |15000.0  |2026-01-11     |45000.0 |
|T103         |C03       |Laptop |1       |50000.0  |2026-01-12     |50000.0 |
+-------------+----------+-------+--------+---------+---------------+-------+
```

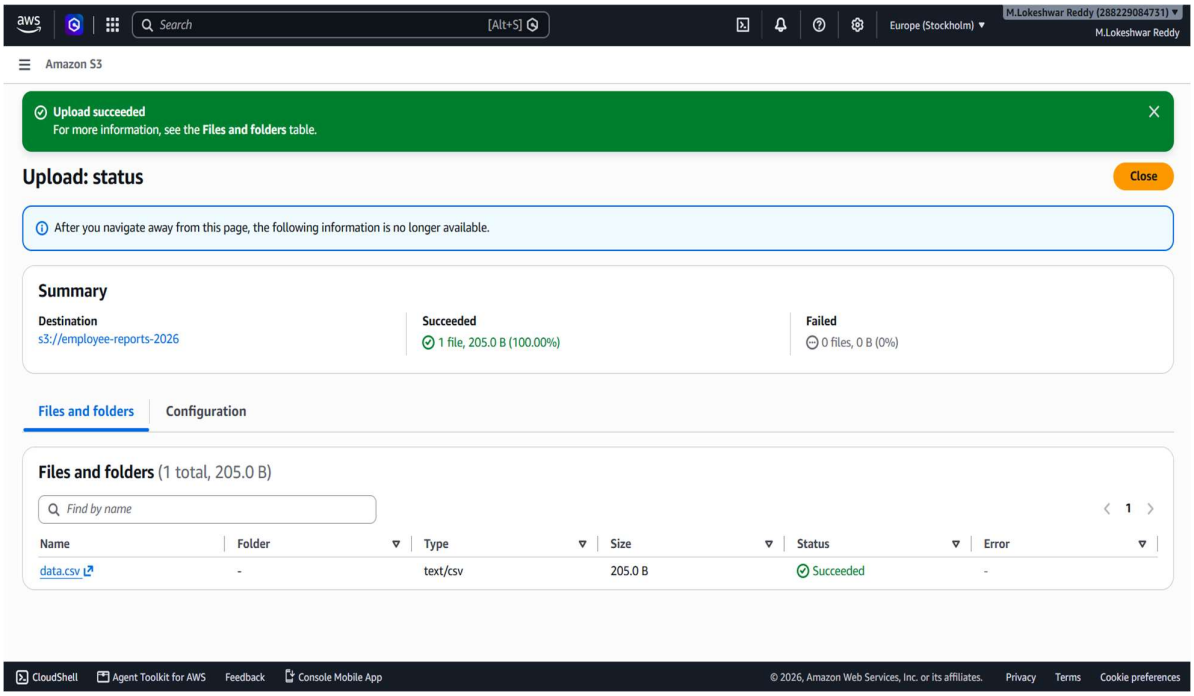
f. Calculate total revenue by product and display the results in descending order.



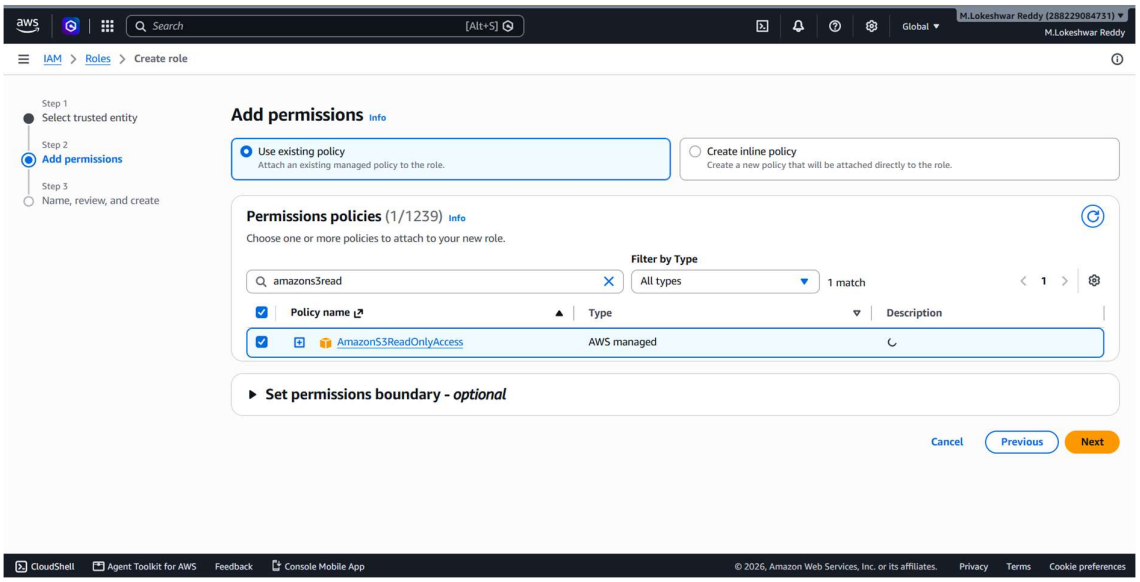
Question 10: S3 and IAM Security



a. Create an S3 bucket with Block Public Access enabled and upload a sample file.



b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.



c. Explain how you would verify that an unauthorized user cannot read the file.

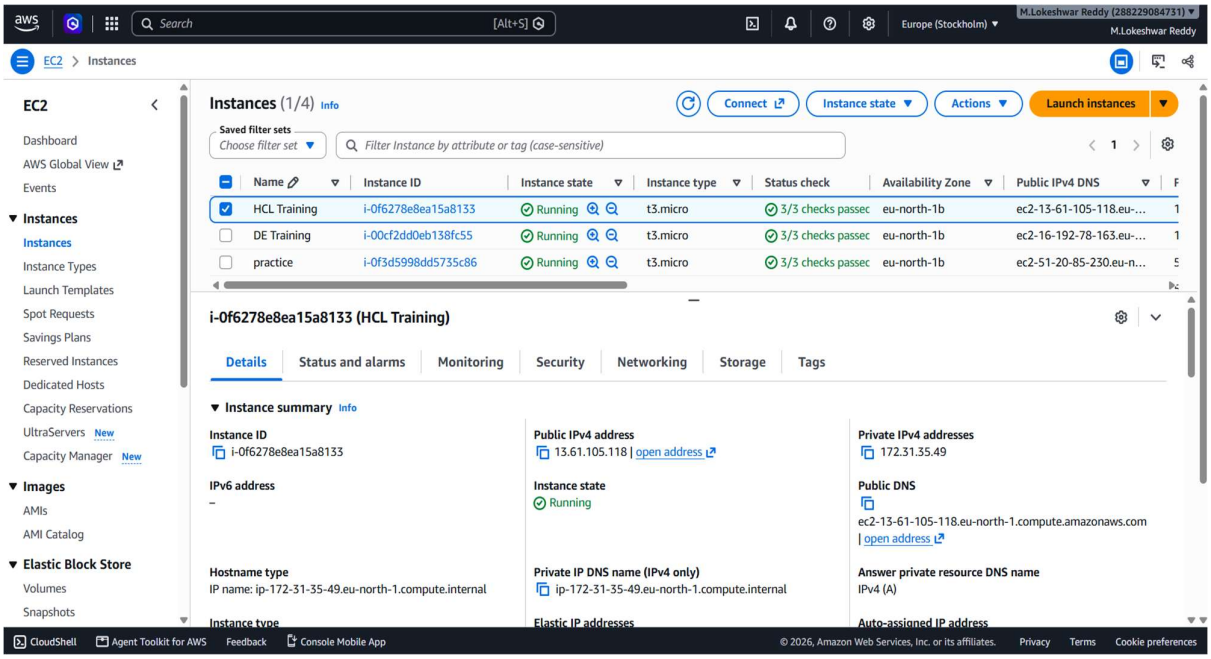
A. Verify file permissions by attempting to access the file using an unauthorized user account. If access is denied, the file is properly protected.

d. Explain how encryption at rest protects the stored data.

A. Encryption at rest protects stored data by converting it into unreadable ciphertext. Only users with the correct decryption key can access the original data.

Question 11: EC2 Troubleshooting and CloudWatch

a. Check the EC2 instance state and status checks.



b. Explain how security groups and route tables can affect application connectivity.

A. Security groups manage the incoming and outgoing traffic, while route tables decide where the network traffic should go. If the rules or routes are configured incorrectly, applications may not be able to connect properly.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

A. Security groups control the traffic coming into and going out of a network, while route tables determine where that traffic needs to go. If these settings are not configured properly, applications may face connectivity issues.

d. Identify one method for investigating the root cause of high CPU utilization.

A. Check Amazon CloudWatch metrics to find sudden increases in CPU usage. Then, review the application logs and running processes to identify which process or task is using too much CPU.

Question 12: Event-Driven Processing

a. Identify a suitable service for executing the processing code.

A. Use AWS Lambda to execute the code that validates and processes the uploaded CSV files.

b. Explain how an S3 upload can trigger processing.

ASet up an Amazon S3 event notification for the ObjectCreated event. This will automatically trigger the Lambda function whenever a CSV file is uploaded to the input S3 bucket.

c. Explain the IAM permissions needed to read the input file and write the results.

A. Create an IAM execution role for Lambda with the necessary permissions:

- s3:GetObject to read input CSV files.
- s3:PutObject to write processed results to the output S3 bucket**.**

d. Explain how errors and processing logs can be captured.

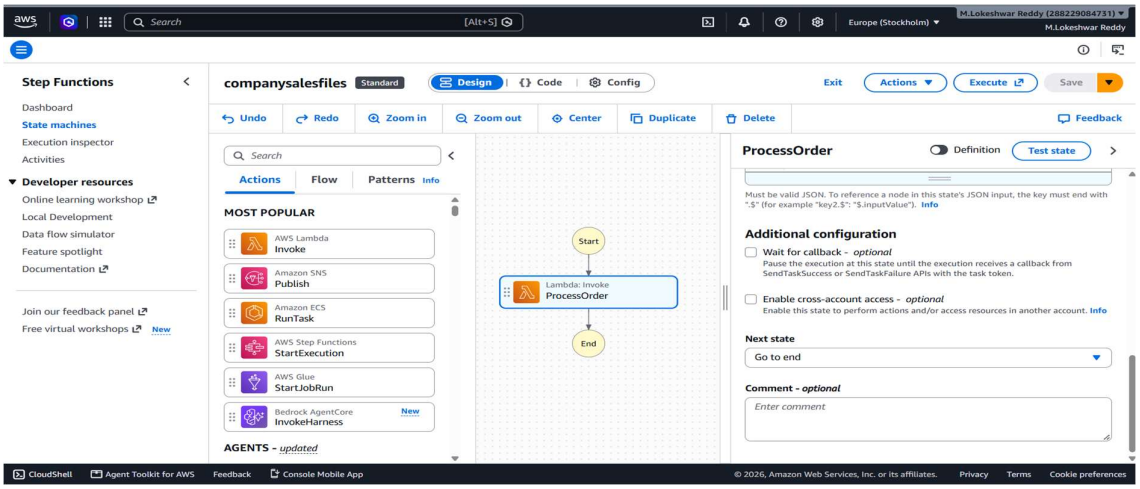
A. Use Amazon CloudWatch Logs to monitor Lambda executions and identify any errors. Set up retries and a Dead-Letter Queue (DLQ) when needed so that failed events can be handled properly.

e. Explain how to prevent duplicate processing or recursive triggering.

A. To avoid processing the same file multiple times, use an idempotency mechanism. To prevent recursive triggering, store the results in a separate output S3 bucket or configure the event notifications to trigger only for CSV files uploaded to the input folder.

Question 13: State Machine Design

a. Identify suitable states for validation, processing, storage, and notification.



b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.

A. The Choice state checks whether the validation was successful. If isValid is true, the workflow moves on to the data transformation step. If it is false, the workflow follows the failure path, where the error is logged and the administrator is notified.

c. Explain how Retry and Catch can handle transient failures and permanent errors.

A. Retry: Automatically retries a failed task caused by transient errors, such as temporary service failures or throttling.Catch: Handles errors that remain after retries by routing execution to an error-handling state or a Fail state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services.

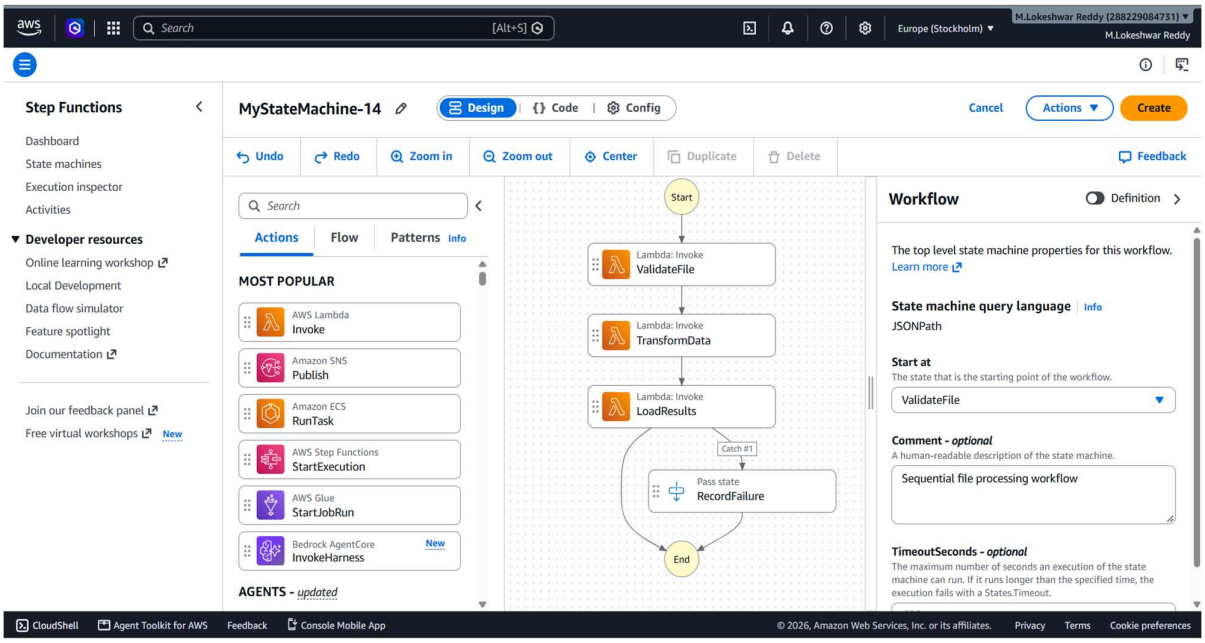
A. Use Task states in AWS Step Functions to run Lambda functions or connect with services like Amazon S3 to store results and Amazon SNS to send notifications. Make sure the state machine's IAM execution role has all the necessary permissions to access these services.

e. Explain how execution history helps troubleshoot a failed workflow.

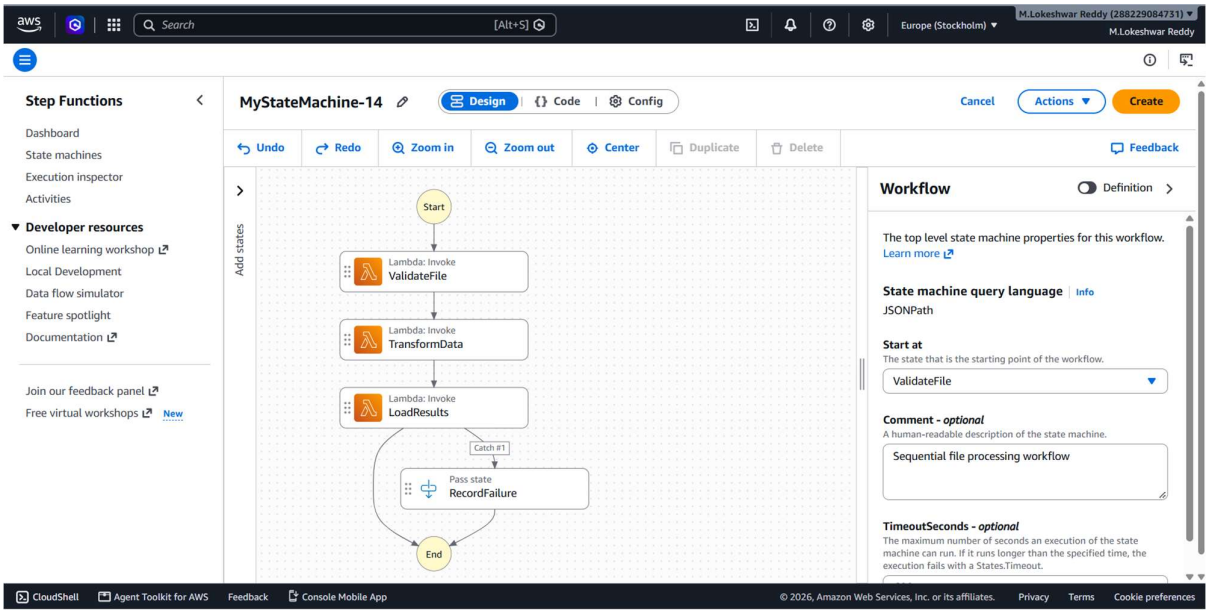
A. AWS Step Functions keeps a record of each execution, including state transitions, inputs and outputs, timestamps, and error messages. Developers can review these details in the AWS Console to find the state where the workflow failed and troubleshoot the issue.

Question 14: Step Functions Implementation

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.



b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts.



c. Configure a Catch transition for the loading task that routes to an error-handling state.

A. The Catch block in LoadResults catches errors and moves execution to HandleError, where the failure is handled.

d. Explain how data from one task is passed to the next task.

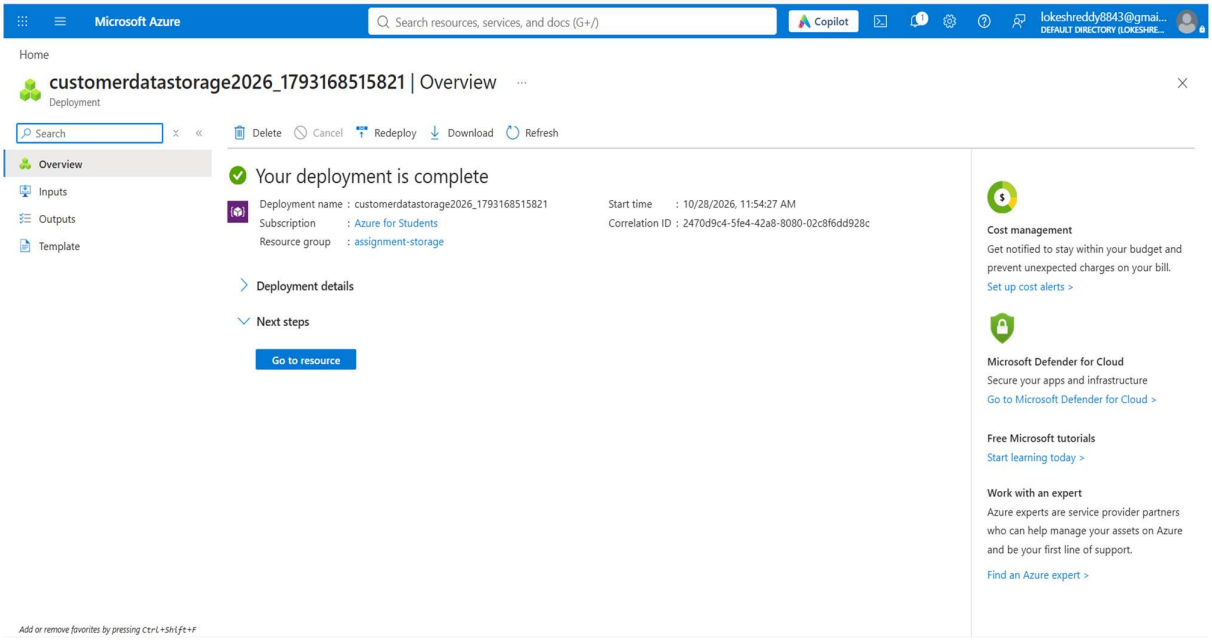
A. Step Functions automatically passes the output of one task as the input to the next task by default. For example, ValidateFile passes its output to TransformData

e. Explain how you would test successful and failed executions.

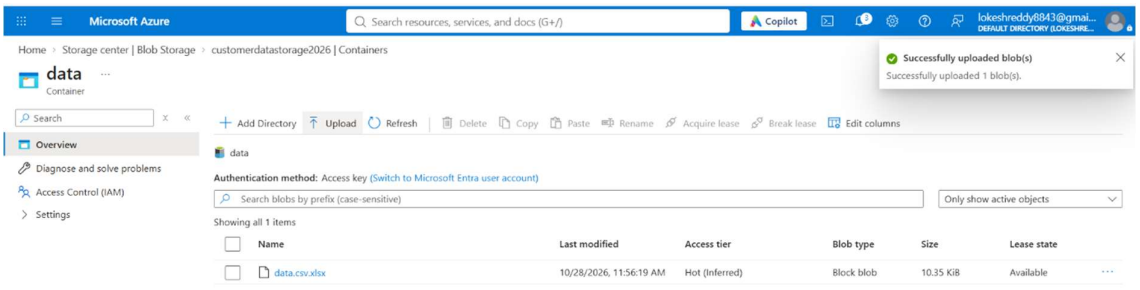
A. Successful test that Provide valid input and check that all three tasks complete successfully. And Failed test Provide invalid input or make LoadResults fail, then check the execution history to verify the failure handling.

Question 15: Azure Data Lake Storage and Security

a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2.



b. Create a filesystem or container and upload a sample dataset.



c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data.

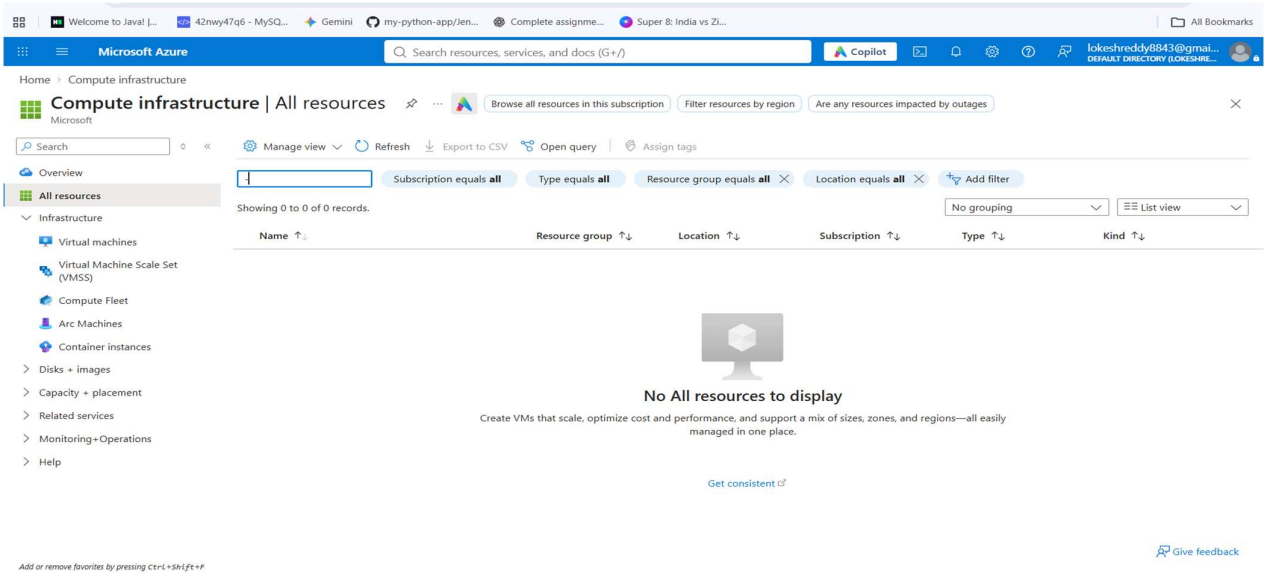
A. Microsoft Entra ID is used for Authenticates users and applications. And Azure RBAC is Assigns permissions based on roles, such as *Storage Blob Data Reader* for reading data and *Storage Blob Data Contributor* for reading, writing, and deleting data.

d. Explain how you would verify that unauthorized users cannot access the dataset.

A. Sign in as a user who has no data-access permissions and try to access the dataset. Confirm that access is denied. Also, verify that the container is private and that anonymous access is disabled.

Question 16: Azure VM Troubleshooting

a. Verify the VM's provisioning and power states.



b. Explain how Network Security Group rules can prevent connectivity.

A. NSG rules may block connectivity if inbound traffic is denied or the required port is not allowed.

c. Explain how subnet and routing configuration affect connectivity

A. Incorrect subnet settings, route tables, or User Defined Routes (UDRs) can prevent traffic from reaching the VM. Check that the subnet is correctly configured and that routes allow traffic to reach the destination.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

A. Use Azure Bastion to connect securely to the VM through the Azure Portal without exposing the VM's public IP address. And Use Network Watcher tools, such as IP flow verify and connection troubleshoot, to identify blocked ports and connectivity issues.

Question 17: Azure Data Integration

a. Select an appropriate service to orchestrate data movement.

A. Use Azure Data Factory to orchestrate and automate data movement and processing.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2.

A.Create a Copy activity in Azure Data Factory to read the CSV file from the source and copy it into Azure Data Lake Storage Gen2 (ADLS Gen2)

c. Describe how to organize files by ingestion date.

A.Store files in date-based folders, for example:/transactions/year=2026/month=10/day=28/

d. Explain how to handle malformed records and missing values.

A. Use Azure Data Factory mapping data flows or data validation steps to identify malformed records and handle missing values. Store invalid records in a separate error folder for further review.

e. Explain how to monitor pipeline execution and validate data completeness.

A. Use Azure Data Factory Monitor to track pipeline execution and errors. Validate data completeness by comparing source and destination record counts and checking for missing or duplicate records.